

A.1 LEXICAL AND CONCRETE SYNTAX OF CLITE

Program $\rightarrow$ int main () { *Declarations Statements* }
Declarations $\rightarrow$ { *Declaration* }
Declaration $\rightarrow$ *Type Identifier* [[*Integer*]] { , *Identifier* [[*Integer*]] } ;
Type $\rightarrow$ int | bool | float | char
Statements $\rightarrow$ { *Statement* }
Statement $\rightarrow$; | *Block* | *Assignment* | *If Statement* | *While Statement*
Block $\rightarrow$ { *Statements* }
Assignment $\rightarrow$ *Identifier* [[*Expression*]] = *Expression* ;
If Statement $\rightarrow$ if (*Expression*) *Statement* [else *Statement*]
While Statement $\rightarrow$ while (*Expression*) *Statement*

Expression $\rightarrow$ *Conjunction* { || *Conjunction* }
Conjunction $\rightarrow$ *Equality* { && *Equality* }
Equality $\rightarrow$ *Relation* [*EquOp Relation*]
EquOp $\rightarrow$ == | !=
Relation $\rightarrow$ *Addition* [*RelOp Addition*]
RelOp $\rightarrow$ < | <= | > | >=
Addition $\rightarrow$ *Term* { *AddOp Term* }
AddOp $\rightarrow$ + | -
Term $\rightarrow$ *Factor* { *MulOp Factor* }
MulOp $\rightarrow$ * | / | %
Factor $\rightarrow$ [*UnaryOp*] *Primary*
UnaryOp $\rightarrow$ - | !
Primary $\rightarrow$ *Identifier* [[*Expression*]] | *Literal* | (*Expression*)
 | *Type* (*Expression*)

Identifier $\rightarrow$ *Letter* { *Letter* | *Digit* }
Letter $\rightarrow$ a | b | ... | z | A | B | ... | Z
Digit $\rightarrow$ 0 | 1 | ... | 9
Literal $\rightarrow$ *Integer* | *Boolean* | *Float* | *Char*
Integer $\rightarrow$ *Digit* { *Digit* }
Boolean $\rightarrow$ true | false
Float $\rightarrow$ *Integer* . *Integer*
Char $\rightarrow$ ' *ASCIICChar* '

Meaning Rule A.10 *The meaning of an ArrayDecl ad is:*

- 1 Compute $\text{addr}(\text{ad}[0]) = \text{new}(\text{ad.size})$, where the value of the Expression ad.size is computed as described in Chapter 7.
- 2 Push $\text{addr}(\text{ad}[0])$ onto the stack.
- 3 Push ad.size onto the stack.
- 4 Push ad.type onto the stack.

Meaning Rule A.11 *The meaning of an ArrayRef ar is:*

- 1 Compute $\text{addr}(\text{ad}[\text{ar.index}]) = \text{addr}(\text{ad}[0]) + \text{ad.index} - 1$.
- 2 If $\text{addr}(\text{ad}[0]) \leq \text{addr}(\text{ad}[\text{ar.index}]) < \text{addr}(\text{ad}[0]) + \text{ad.size}$, return the value at $\text{addr}(\text{ad}[\text{ar.index}])$.
- 3 Otherwise, signal an index-out-of-range error.

The meaning of an Assignment as whose target is an ArrayReference ar is:

Meaning Rule A.12 *The meaning of an Assignment as is:*

- 1 Compute $\text{addr}(\text{ad}[\text{ar.index}]) = \text{addr}(\text{ad}[0]) + \text{ad.index} - 1$.
- 2 If $\text{addr}(\text{ad}[0]) \leq \text{addr}(\text{ad}[\text{ar.index}]) < \text{addr}(\text{ad}[0]) + \text{ad.size}$, then change the value at $\text{addr}(\text{ad}[\text{ar.index}])$ to the value of as.source .
- 3 Otherwise, signal an index-out-of-range error.

A.5 ADDING FUNCTIONS TO CLITE

A.5.1 Lexical and Concrete Syntax

In this definition, underlining indicates additions to the original rules for Clite given in Section A.1.

Program $\rightarrow \{ \text{Type Identifier FunctionOrGlobal} \} \text{MainFunction}$

Type $\rightarrow \text{int} \mid \text{boolean} \mid \text{float} \mid \text{char} \mid \underline{\text{void}}$

FunctionOrGlobal $\rightarrow (\text{Parameters}) \{ \text{Declarations Statements} \} \mid \text{Global}$

Parameters $\rightarrow [\text{Parameter} \{ , \text{Parameter} \}]$

Parameter $\rightarrow \text{Type Identifier}$

Global $\rightarrow \{ , \text{Identifier} \} ;$

MainFunction $\rightarrow \text{int main} () \{ \text{Declarations Statements} \}$

Statement $\rightarrow ; \mid \text{Block} \mid \text{Assignment} \mid \text{IfStatement}$

WhileStatement $\mid \text{CallStatement} \mid \text{ReturnStatement}$

CallStatement $\rightarrow \text{Call};$

ReturnStatement $\rightarrow \text{returnExpression};$

Factor $\rightarrow \text{Identifier} \mid \text{Literal} \mid (\text{Expression}) \mid \underline{\text{Call}}$

Call $\rightarrow \text{Identifier} (\text{Arguments})$

Arguments $\rightarrow [\text{Expression} \{ , \text{Expression} \}]$